PYTHON / HUGGING FACE

자유주제 AI 챗봇 기획서

2026년 11월 9일 시작 · 기본 2주 · 확장 최대 4주

01 범위와 현재 상태

Python과 Hugging Face를 활용해 자유주제 한국어 챗봇을 개발한다. 모델 로딩, 터미널 연속 대화, Gradio 화면은 구현 완료 상태다. 이후 일정은 답변 품질 개선과 실행 재현에 집중한다.

월·수·금 작업을 가정한다. 기본 기간은 11월 9~22일, 확장 기간은 11월 23일~12월 6일이다. 표기한 날짜 사이에는 재실행과 지연에 대응할 여유를 둔다.

02 1주차 · 기준 모델 평가

11/09 월 · 현재 버전 기록

app.py와 web_app.py를 실행한다. 모델 ID, Python·패키지 버전, CPU/GPU, 시스템 메시지, 대화 3쌍, 생성 토큰 한도를 기록한다. 결과물은 기준 설정표다.

11/11 수 · 평가 사례 작성

일상 대화, 사실·개념, 요약·형식, 후속 대화에서 각각 5개씩 만든다. 입력과 통과 조건을 먼저 고정한다. 예를 들어 '세 가지'에는 개수가 정확한지 채점한다.

11/13 금 · 기준값 측정

현재 모델로 20개 사례를 실행한다. 응답을 저장하고 사실 오류, 지시 누락, 맥락 누락, 잘림을 구분한다. 초기 로딩과 응답 생성 시간을 분리하고, 최대 관측 메모리를 적는다.

03 2주차 · 개선과 기본 버전 확정

11/16 월 · 설정 한 가지 조정

잘림이 많으면 max_new_tokens를, 형식 누락이 많으면 시스템 메시지를 조정한다. 실패 사례와 통과 사례를 재실행해 개선과 회귀를 함께 기록한다.

11/18 수 · 모델 후보 1개 비교

RAM과 실행 장치에 맞는 Instruct 후보를 고른다. 모델과 전용 토크나이저를 함께 바꾸고 2~3개 질문으로 시험한 뒤 고정 사례 20개로 비교한다.

11/20 금 · 기본 버전 마감

품질, 응답 시간, 메모리를 보고 모델과 설정을 선택한다. requirements.txt와 README.md를 작성하고 새 가상환경에서 두 실행 경로를 확인한다.

PYTHON / HUGGING FACE

확장 일정과 평가 기준

3~4주차는 기본 버전에서 해결하지 못한 품질 문제가 있을 때 진행

04 3주차 · 반복 오류 집중 개선

11/23 월 · 실패 재현

반복 실패 하나를 선택한다. 예를 들어 이름을 기억하지 못하면 이름이 모델 입력에 실제로 남아 있는지 확인한다. 같은 증상을 재현할 새 질문을 작성한다.

11/25 수 · 원인별 변경

앞 대화가 잘렸다면 기록 범위를 한 단계 늘린다. 기록이 포함됐는데도 실패하면 모델별 결과를 비교한다. 변경 전후의 품질과 속도, 메모리를 기록한다.

11/27 금 · 새 사례와 회귀 검수

이름·숫자·조건을 바꾼 새 질문으로 개선을 확인한다. 이전에 통과한 질문과 초기화도 다시 실행해 기존 동작을 검수한다.

05 4주차 · 최종 검수와 문서화

11/30 월 · 최종 성능 측정

고정 사례 20개를 재실행해 최초 모델과 유형별 통과 수를 비교한다. 개선된 답변과 남은 오류 예시를 각각 기록한다.

12/02 수 · 새 환경에서 설치 검수

새 가상환경에서 requirements.txt를 설치하고 app.py와 web_app.py를 실행한다. 빈 입력, 후속 질문, 초기화를 확인한다.

12/04 금 · 최종 산출물 정리

코드 2개, 의존성 목록, README, 성능 비교표를 정리한다. 최종 모델 ID와 설정, 선택 이유, 알려진 한계를 기록한다.

06 모델 선택 기준

같은 PC에서 모델 로딩 후 동일한 질문을 사용한다. 사례별로 새 대화를 시작하고, 후속 질문은 같은 세트 안에서 이어간다. 통과 수와 생성 시간, 최대 관측 메모리를 함께 비교한다.

PYTHON / HUGGING FACE

기술별 적용 방식

개발 환경부터 대화 생성과 검수까지

07 개발 환경과 모델 관리

VS Code · 코드 편집과 디버깅

프로젝트 폴더를 열고 Python: Select Interpreter에서 .venv를 선택한다. 통합 터미널에서 앱을 실행하고 필요하면 답변 함수에 중단점을 걸어 메시지 역할과 개수를 확인한다.

Python · venv · pip · 실행과 의존성

Python에서 질문, 최근 기록, 답변을 처리한다. venv로 의존성을 분리하고 .venv의 python.exe -m pip로 설치한다. 재현에 필요한 패키지 버전은 requirements.txt에 기록한다.

Hugging Face Hub · 모델 배포 저장소

Qwen/Qwen2.5-0.5B-Instruct를 기준으로 모델 카드에서 실행 예제와 요구사항을 확인한다. from_pretrained()로 가중치와 토크나이저를 받고, 비교 모델의 ID도 기록한다.

08 답변 생성

Transformers · 모델 입력과 출력

AutoTokenizer와 AutoModelForCausalLM으로 모델을 불러온다. 시스템 메시지와 최근 대화, 질문을 apply_chat_template()으로 포맷한다. generate() 결과에서 입력 길이만큼 제외하고 새 토큰만 디코딩한다.

PyTorch · 모델 연산

가중치와 입력 텐서를 같은 장치에 둔다. 현재 실행 장치에서 먼저 비교하며, 평가 시 model.eval()을 유지한다. 응답 생성은 torch.inference_mode()로 감싸 추론 중 기울기 계산을 막을 수 있다.

Qwen 모델 카드
Hugging Face 채팅 템플릿

PYTHON / HUGGING FACE

화면 연결과 실행 검수

터미널과 웹에서 동일한 답변 생성 코드 사용

09 사용자 입력과 대화 기록

app.py · 터미널 입력과 공통 생성 함수

입력의 앞뒤 공백을 제거하고 빈 입력을 건너뛴다. /reset은 기록을 비우고 /exit는 반복을 종료한다. 일반 질문은 최근 질문·답변 3쌍과 함께 generate_reply()에 넘긴다.

web_app.py · Gradio ChatInterface

현재 질문과 화면의 대화 기록을 콜백에서 받는다. 텍스트 메시지의 role과 content를 추려 generate_reply()에 전달한다. launch(share=False)로 로컬 화면을 열고, 기록을 지운 후 초기화를 검수한다.

10 측정과 완료 기준

Python time · 응답 시간

time.perf_counter()로 모델 로딩과 답변 생성을 따로 측정한다. 모델 비교 시 첫 워밍업 응답은 따로 표시하고, 같은 질문 세트에서 생성 시간 중앙값을 비교한다.

Windows 작업 관리자 · 메모리

다른 모델 프로세스를 종료하고 모델 하나씩 실행한다. Python 프로세스의 RAM을 같은 방식으로 관찰해 최대 관측값을 기록한다. GPU 실행 시 전용 GPU 메모리를 따로 기록한다.

검수 기록과 실행 문서

사례마다 모델 ID, 설정, 기대 결과, 실제 답변, 통과 여부를 남긴다. README에 설치, 실행, 첫 모델 다운로드, 알려진 한계를 적는다. 새 환경에서 터미널과 웹의 일반 질문, 후속 질문, 초기화가 동작하면 마감한다.

Gradio ChatInterface 소스